VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Infosys Springboard Internship 7.0

Milestone 1

By Pathlavath Vasu Nayak From *Team -1*

Table of Contents

1 Project Description

The VisionInspect AI system is an end-to-end, enterprise-grade machine learning and computer vision platform designed to automate quality control and defect detection in Industry 4.0 smart manufacturing environments. The primary goal of this project is to replace manual visual inspection with an automated, high-throughput AI pipeline capable of identifying surface anomalies (such as cracks, scratches, and structural deformities) on manufactured products in real time.

Milestone 1 focuses on establishing the core platform infrastructure. This includes designing a full-stack architecture, implementing secure **Role-Based Access Control (RBAC)** via **JSON Web Tokens (JWT)**, integrating standard industrial benchmark datasets, and deploying a live **REST API** ingestion gateway paired with an interactive multi-role frontend dashboard.

```
backend/
│
├── app/
│   ├── api/
│   │   ├── auth.py
│   │   └── endpoints.py
│   │
│   ├── core/
│   │   └── database.py
│   │
│   ├── ml_engine/
│   │
│   ├── models/
│   │   └── db_models.py
│   │
│   ├── utils/
│   │   └── dataset.py
│   │
│   └── main.py
│   ├── output_charts/
│   ├── analytics.ipynb
│   └── main.py
├── visioninspect.db
├── requirements.txt
└── env/
```

Fig 1: Backend project structure showing API, database , model and utility modules

2 Dataset Used

The project utilizes the global industrial benchmark MVTec Anomaly Detection (MVTec AD) dataset.

Original Scope: The dataset contains thousands of high-resolution industrial product images across 15 distinct manufacturing categories (e.g., bottles, capsules, metal nuts, cables, and transistors).

Data Structure: Each product category is divided into clean, defect-free control samples (good training directory) and specific anomaly classes (such as crack, scratch, broken_large, and contamination in the test directory).

Milestone 1 Data Pipeline (utils/dataset.py): To facilitate automated ingestion, a custom PyTorch Dataset and DataLoader utility was engineered.
The loader systematically traverses folder hierarchies, maps defect names to numerical labels (**0** for **good**, **1** for **anomalous**), and applies uniform image standardization (RGB channel conversion and resizing) to prepare raw data for downstream neural network ingestion.

3 Environment Setup

The platform was developed using a decoupled full-stack architecture, executed locally within an isolated Python virtual environment (env) for the backend and a Node.js runtime for the frontend. The implementation was conducted using Python 3.13 and React.js within Visual Studio Code.

Key Libraries and Technical Configurations:

- ***FastAPI & Uvicorn:*** Serves as the high-performance asynchronous backend framework and ASGI server, handling concurrent image stream uploads and CORS middleware communication.

- ***SQLAlchemy & SQLite:*** Manages relational database persistence (visioninspect.db), structurally mapping user authentication credentials and logging comprehensive inspection metadata.

- ***PyJWT & Passlib (Bcrypt):*** Power the authentication security layer, executing SHA-256 password hashing and token-based role authorization.

- ***PyTorch (torch, torchvision):*** Initialized within the data utility module to structure image tensors and batch loading for upcoming ML models.

- ***OpenCV (opencv-python) & Pillow (PIL):*** Essential computer vision libraries utilized for binary stream verification, file format validation, and image dimension extraction.

- ***React.js, Vite, & Tailwind CSS:*** Drive the frontend interactive interface, utilizing modern utility-first styling and axios for HTTP API telecommunications.

Split terminal execution displaying active FastAPI backend (Uvicorn) and Vite React frontend servers.

4 System Architecture & Role-Based Access Control (RBAC)

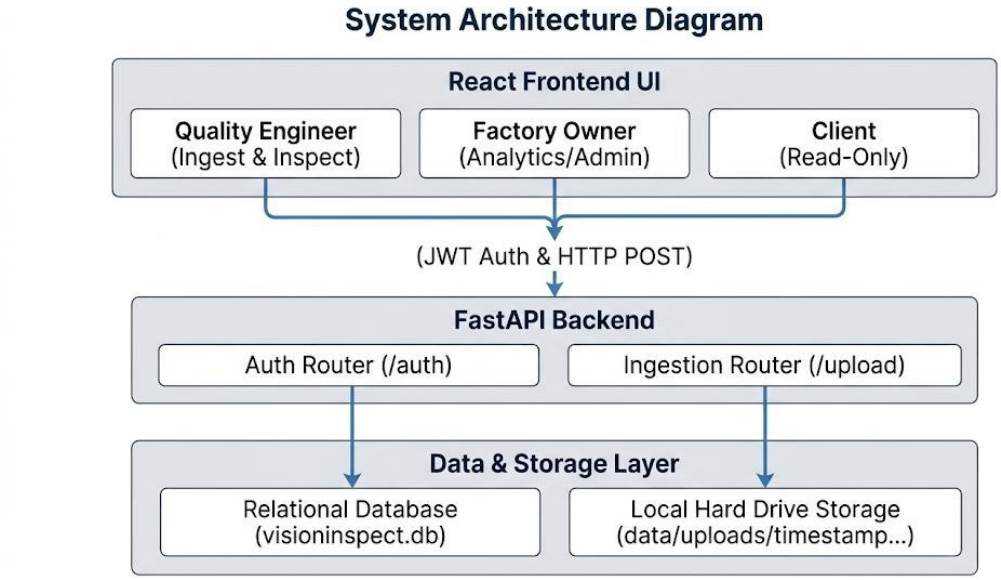


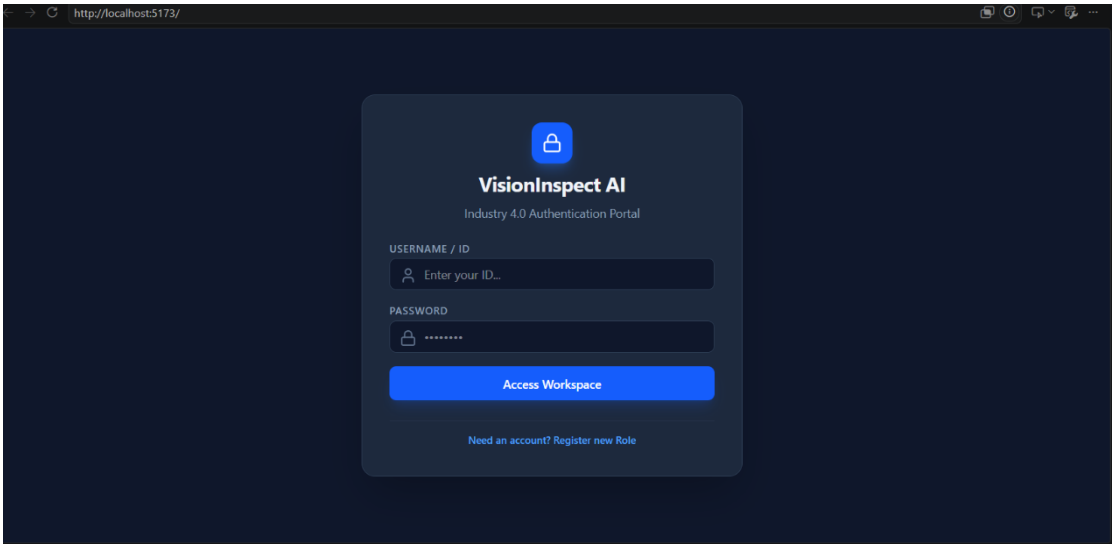
Fig 2 : Backend Architecture and communication flow

1. **Quality Engineer (Active Inspector):** Granted operational permissions to access the image dropzone, ingest product samples, run technical prechecks, and log diagnostic results.

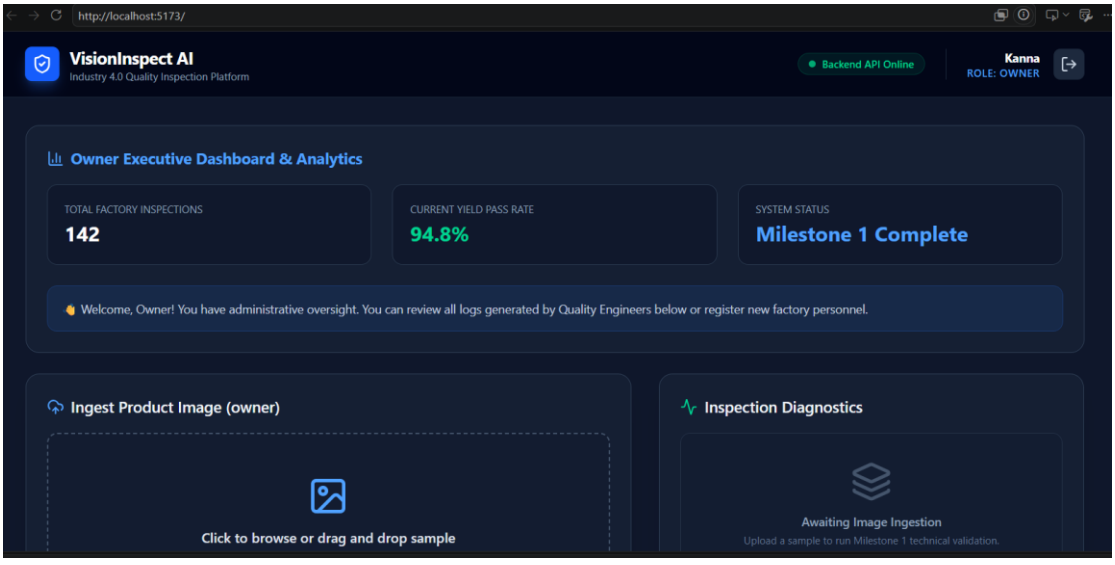
2. **Factory Owner / Supervisor (Executive Admin):** Receives full oversight capabilities, including access to global factory analytics (total inspection volume, yield pass rates), system health telemetry, and historical audit logs.

3. **Client / Buyer (Verification Portal)**: Restricted to a read-only interface to independently verify batch integrity checks and review quality assurance reports without alteration permissions.

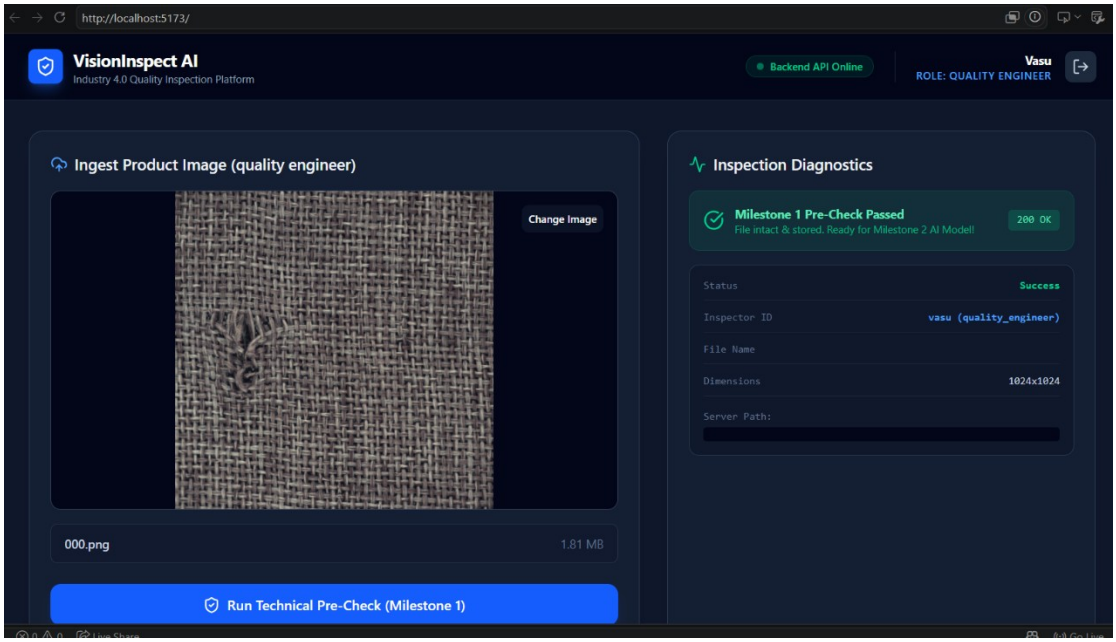
The Above Images Shows About the Dashboard of the Website



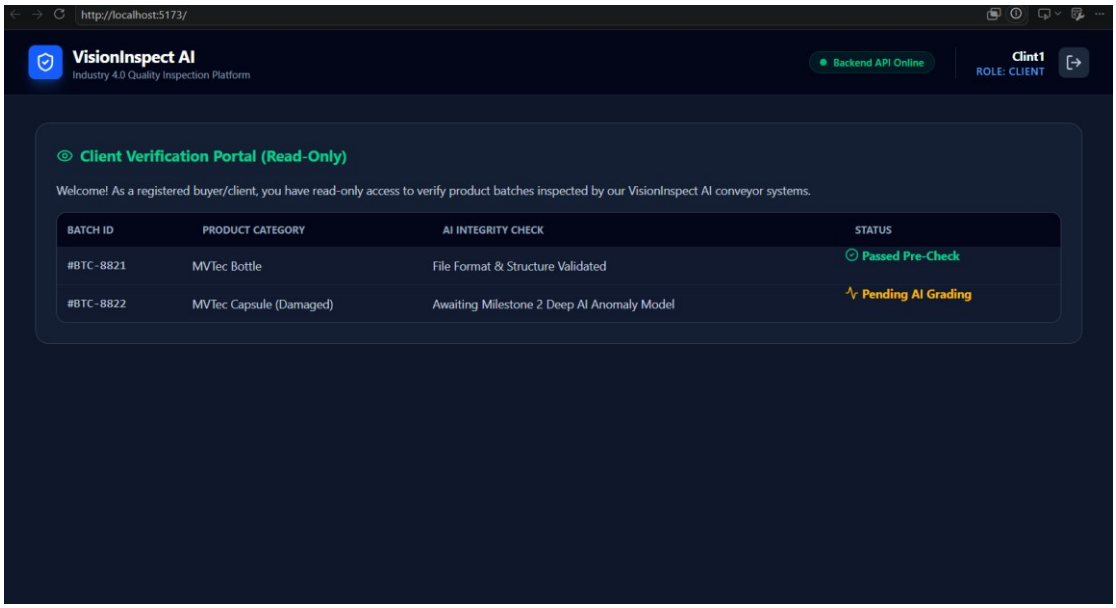
The VisionInspect AI JWT Authentication Portal.



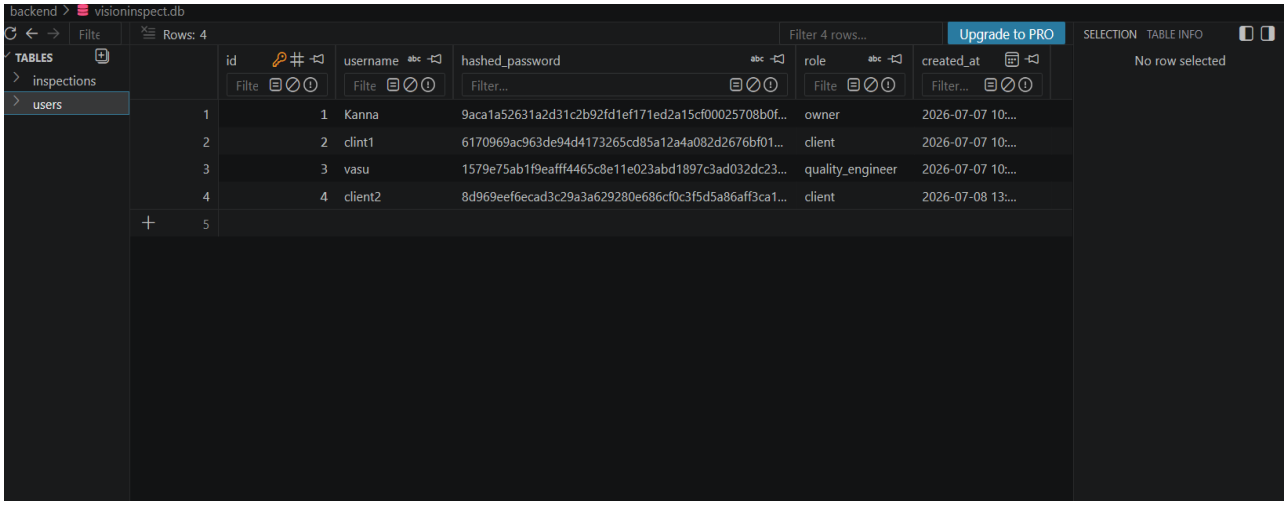
Owner dashboard to check the insight



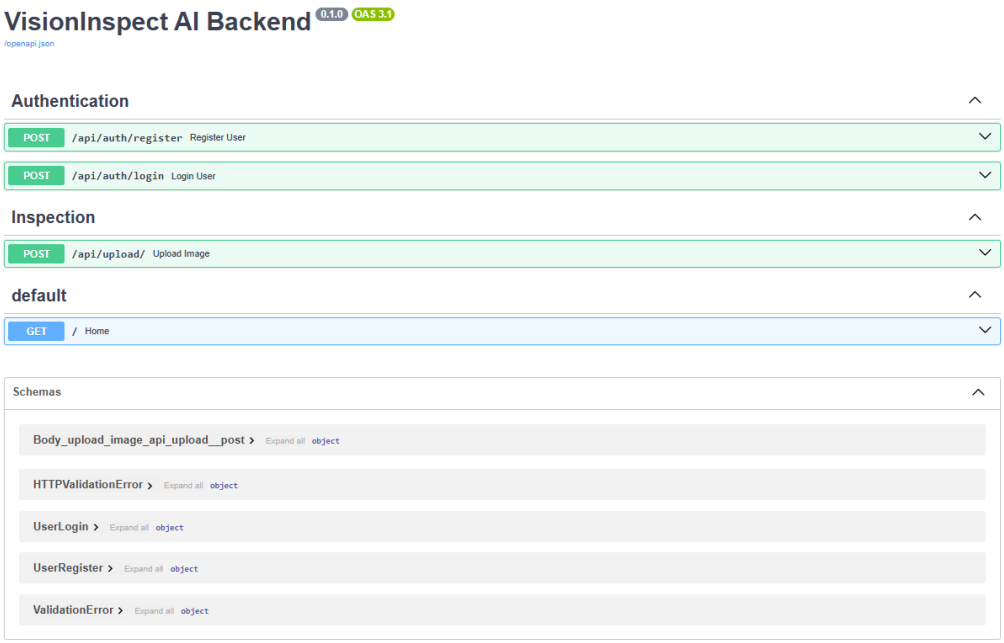
Quailty Engineer Dashboard



Role-Based UI dynamically rendering the Client Verification Portal with read-only batch telemetry.



Visioninspect.db of SQLite Database inspection revealing automated schema generation, role assignment, and encrypted password hashing.



Auto-generated Interactive Swagger API documentation validating backend endpoint routing and multi-part data payload acceptance.

5 Core Implementation: Ingestion Pipeline & Multi-Role UI

The Milestones 1 technical workflow operates through a structured sequence to guarantee data integrity before machine learning analysis:

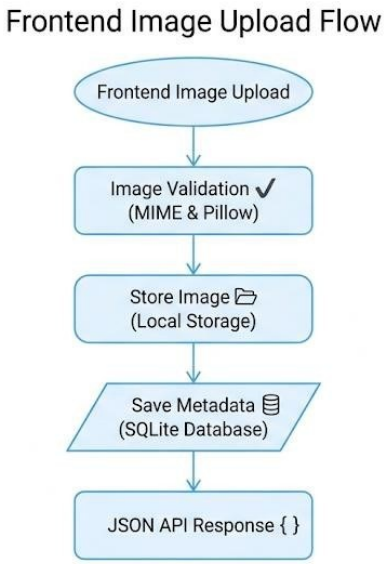


Fig 3 : Image Ingestion and Validation Pipeline

1. **Binary Stream Ingestion:** An image is submitted via the frontend dragand-drop interface or automated conveyor simulator to the POST /api/upload/ endpoint.

2. **MIME & Technical Pre-Check:** The server validates that the incoming stream is a legitimate image format (.png, .jpg, .jpeg), blocking malicious file executions.

3. **Pillow Structural Verification:** The image binary is read into memory and processed through image.verify() to detect corrupted packets or incomplete transmissions.
4. **Timestamped Disk Archiving:** To prevent naming collisions during high-speed factory batch processing, the file is assigned a unique timestamp (YYYYMMDD_HHMMSS_filename) and permanently committed to the data/uploads/ directory.
5. **JSON Telemetry Return:** The server generates and returns a structured payload detailing file dimensions, color architecture, storage

paths, and HTTP status codes (200 OK) to update the UI dashboard in real time.

6 Verification & Technical Pre-Check Results

During formal system verification, the platform was evaluated across multiple operating system parameters and network requests:

- ✓ **Authentication Integrity:** Successfully prevented unauthorized route access and correctly dynamically rendered role-specific workspaces upon JWT token validation.

- ✓ **File Ingestion Rigor:** Tested against valid industrial samples (015.png, 001.png) and intentionally damaged or corrupted files. The backend successfully committed valid files to disk while returning 400 Bad Request exceptions for invalid file formats.

- ✓ **Database Generation:** Verified the automated generation and schema population of users and inspections tables within the SQLite engine upon application boot (create_all).

7 Conclusion and Next Steps

Milestone 1 successfully establishes the foundational data, web, and security architecture required for an enterprise-grade Industry 4.0 inspection platform. By bridging React frontend interfaces with an asynchronous FastAPI backend and automated PyTorch data loaders, the platform reliably ingests, validates, and stores industrial image streams.